

Experiment No. – 6							
Date of Performance:	1st Aug 2026						
Date of Submission:	1st Aug 2026						
Conceptual Understanding (02)	Execution (1.5)	Problem Solving & Debugging(02)	Professional Conduct & Lab Ethics(02)	Output/ Result Accuracy (1.5)	Documentation & Result Analysis (01)	Experiment Total (10)	Sign with Date

Experiment No. 6 Study of Inheritance

6.1 Aim: Implementation of Multi-level inheritance using a 3-class hierarchy : WAP in Java to implement the class hierarchy Where Student class has name and roll_no, Test class has subject 1 and subject 2 marks, Sports is a class which has score of 5 marks a constant data member, and a Result class will display the total marks of student.

6.2 Course Outcome: Implement inheritance, interfaces, and packages to achieve code reusability and modular program design.

6.3 Learning Objectives: As java does not support multiple inheritance but there are occasions where a class needs to access data and methods from two or more classes in such cases Interfaces can be used and multiple inheritance can be achieved in java.

6.4 Requirement: Java development kit (JDK as per OS)

6.5 Related Theory:

Inheritance is one of the fundamental concepts of **Object-Oriented Programming (OOP)**. It enables one class (called the **subclass** or **derived class**) to inherit the properties and methods of another class (called the **superclass** or **base class**). This promotes **code reusability**, reduces redundancy, and establishes an **"IS-A" relationship** between classes.

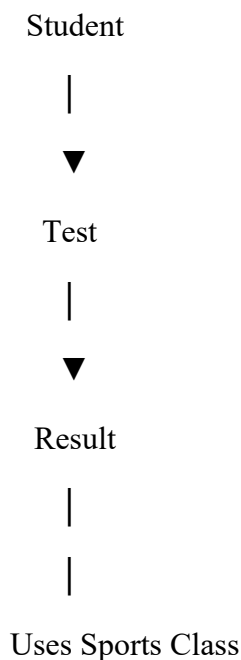
In Java, inheritance is achieved using the **extends** keyword. A subclass inherits all accessible data members and methods of its superclass and can also add new members or override inherited methods.

In this experiment, a **Student** class acts as the base class and stores the student's name and roll number. A **Test** class extends the Student class and stores marks of two subjects. A **Sports** class stores a constant sports score. Finally, a **Result** class extends the Test class and uses the sports score to calculate and display the total marks of the student.

This experiment demonstrates:

- Single inheritance
- Multilevel inheritance
- Use of constructors
- Constant (final) data member
- Code reusability through inheritance

Inheritance Hierarchy



Advantages of Inheritance

- Promotes code reusability.
- Reduces duplicate code.
- Makes programs modular and easy to maintain.
- Supports method overriding.
- Represents real-world relationships.

Types of Inheritance in Java

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (through Interfaces)
5. Hybrid Inheritance (using Interfaces)

Keywords Used

Keyword	Purpose
extends	Used to inherit one class from another.
super	Calls superclass constructor or members.
final	Declares constants that cannot be modified.

6.6 Procedure:

Create a Java program by first defining a **Student** class containing the student's name and roll number. Next, create a **Test** class that extends the **Student** class and includes the marks of two subjects. Then, create a **Sports** class containing a constant data member representing the sports score. After that, create a **Result** class that extends the **Test** class and accesses the sports score from the **Sports** class to calculate the total marks of the student. Instantiate an object of the **Result** class in the **main()** method, initialize the required data using constructors or methods, calculate the total marks by adding the subject marks and sports score, and display the student's details along with the total marks. Finally, compile the program, execute it, and verify the output.

6.7 Program and Output:

```
import java.util.Scanner;
```

```
class Student {  
    String name;
```

```
int rollNo;
```

```
void getStudentData(Scanner sc) {  
    name = sc.nextLine();  
    rollNo = sc.nextInt();  
}  
}
```

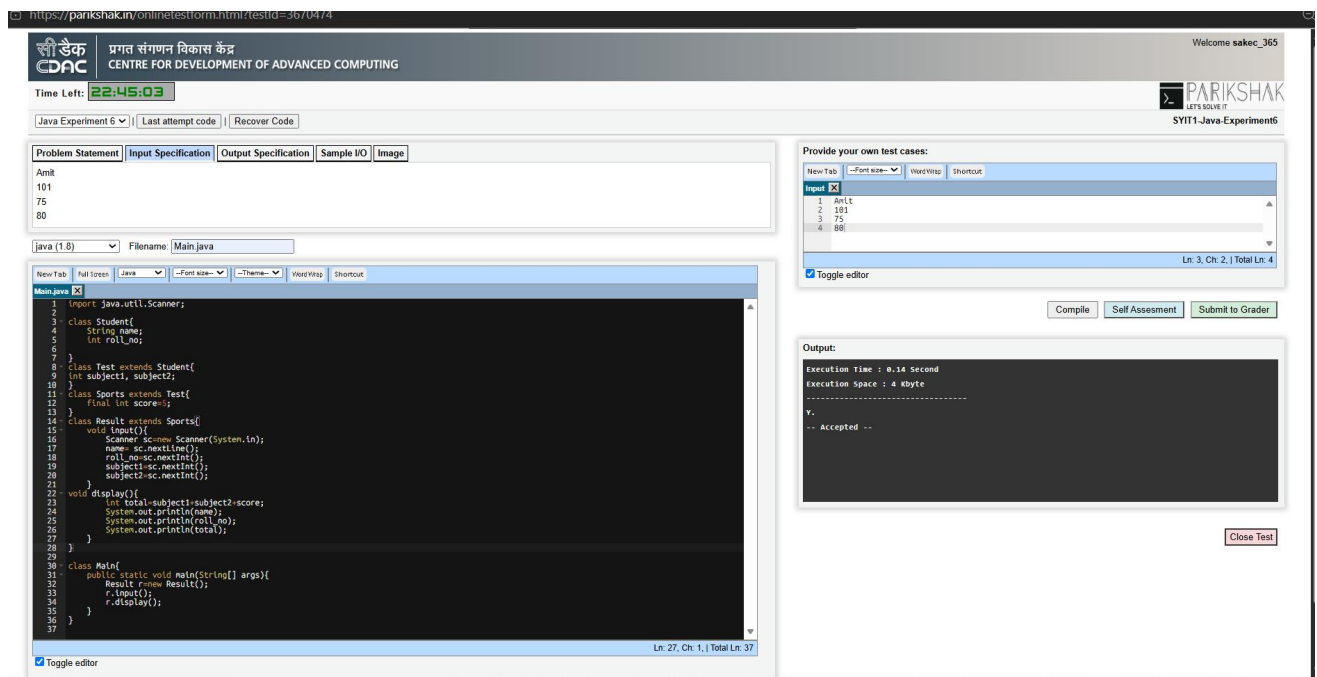
```
class Test extends Student {  
    int subject1, subject2;  
  
    void getMarks(Scanner sc) {  
        subject1 = sc.nextInt();  
        subject2 = sc.nextInt();  
    }  
}
```

```
class Sports extends Test {  
    final int sportsScore = 5;  
}
```

```
class Result extends Sports {  
    void displayResult() {  
        int total = subject1 + subject2 + sportsScore;  
  
        System.out.println("Student Name : " + name);  
        System.out.println("Roll Number : " + rollNo);  
    }  
}
```

```
        System.out.println("Subject 1   : " + subject1);  
        System.out.println("Subject 2   : " + subject2);  
        System.out.println("Sports Score : " + sportsScore);  
        System.out.println("Total Marks : " + total);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        Result r = new Result();  
  
        r.getStudentData(sc);  
        r.getMarks(sc);  
  
        r.displayResult();  
  
        sc.close();  
    }  
}
```



6.8 Conclusion:

This experiment demonstrated the concept of multilevel inheritance in Java. The Result class inherited the properties and methods of the Student, Test, and Sports classes to calculate the total marks of a student. The program showed how inheritance promotes code reusability, reduces redundancy, and makes programs modular and easier to maintain. It also illustrated the use of a constant (final) data member for the sports score.

6.9 Questions:

1. What is inheritance in Java? Explain its advantages and the different types of inheritance supported in Java.

Inheritance is an Object-Oriented Programming (OOP) feature in Java that allows one class to acquire the properties and methods of another class using the extends keyword. It establishes an IS-A relationship between classes.

Advantages:

- Promotes code reusability.
- Reduces code duplication.
- Makes programs modular and easy to maintain.
- Supports method overriding and runtime polymorphism.
- Improves readability and extensibility of programs.

Types of Inheritance in Java:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (through interfaces only)
5. Hybrid Inheritance (using interfaces)

-
2. Differentiate between a superclass and a subclass. Explain the role of the extends keyword in implementing inheritance with a suitable example.

Superclass	Subclass
Parent or base class.	Child or derived class.
Provides data members and methods.	Inherits and uses the members of the superclass.
Can exist independently.	Depends on the superclass for inherited features.

The extends keyword is used to inherit one class from another. It allows the subclass to access the public and protected members of the superclass, promoting code reuse.

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // Inherited method  
        d.bark(); // Dog's own method  
    }  
}
```

Output:

Animal is eating

Dog is barking

.